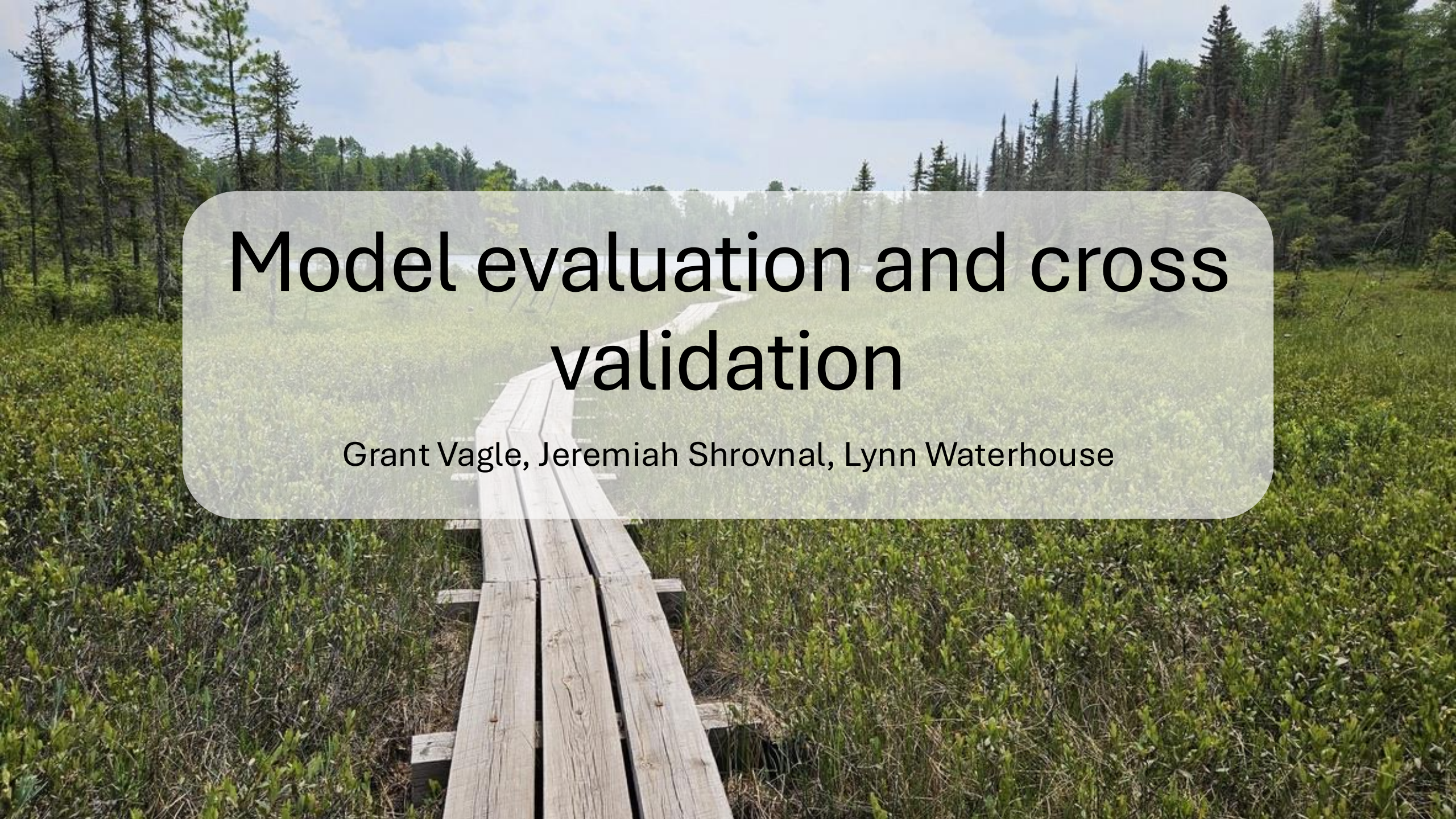


Model evaluation and cross validation

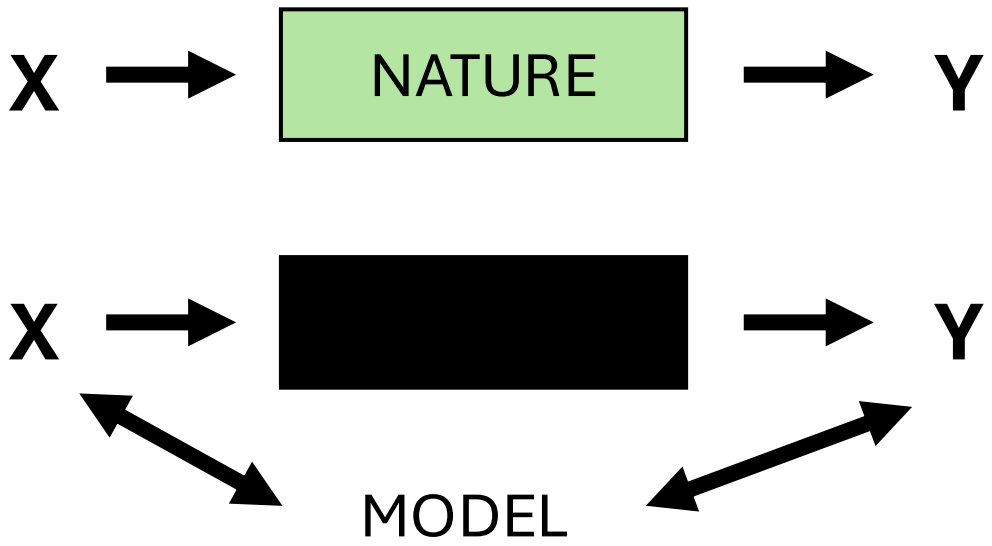
Grant Vagle, Jeremiah Shrovnal, Lynn Waterhouse

Overview of this section

- Understanding how well a model is performing
  - Performance metrics
  - Evaluation / cross-validation
- Introduce Classification and Regression Trees (CART)
- Hands-on coding example

How do we measure how well a model mimics the outputs of nature?

1. Model performance metrics – *what* we calculate
2. Model validation – *which data* we use to make the calculation



Molnar, C. 2023. Modeling mindsets: the many cultures of learning from data.

Model performance metrics - classification

Many (most?) performance metrics for classification come from the **confusion matrix**

Let's say we have a model that predicts a yes or no response

	Observed response	Predicted response
1	Yes	Yes
2	No	Yes
3	Yes	Yes
…		
100	Yes	No

Model performance metrics - classification

Many (most?) performance metrics for classification come from the **confusion matrix**

Let's say we have a model that predicts a yes or no response

	Observed response	Predicted response
1	Yes	Yes
2	No	Yes
3	Yes	Yes
…		
100	Yes	No

Confusion matrix:

	Predicted Yes	Predicted No
Observed Yes		
Observed No		

Model performance metrics - classification

Many (most?) performance metrics for classification come from the **confusion matrix**

Let's say we have a model that predicts a yes or no response

	Observed response	Predicted response
1	Yes	Yes
2	No	Yes
3	Yes	Yes
…		
100	Yes	No

Confusion matrix:

	Predicted Yes	Predicted No
Observed Yes	42	4
Observed No	18	36

Model performance metrics - classification

	Predicted Yes	Predicted No
Observed Yes	42	18
Observed No	4	36

Model performance metrics - classification

	Predicted Yes	Predicted No
Observed Yes	42	18
Observed No	4	36

$$\text{Accuracy} = \frac{\#\ \text{correct}}{\#\ \text{correct} + \#\ \text{incorrect}}$$

Model performance metrics - classification

	Predicted Yes	Predicted No
Observed Yes	42	18
Observed No	4	36

$$\text{Accuracy} = \frac{\#\ \text{correct}}{\#\ \text{correct} + \#\ \text{incorrect}}$$ = (42+36) / 100 = 78%

Model performance metrics - classification

	Predicted Yes	Predicted No
Observed Yes	42	18
Observed No	4	36

$$\text{Accuracy} = \frac{\#\ \text{correct}}{\#\ \text{correct} + \#\ \text{incorrect}}$$ = (42+36) / 100 = 78%

Elements of the confusion matrix are **not** created equal:

Model performance metrics - classification

	Predicted Yes	Predicted No
Observed Yes	42	18
Observed No	4	36

$$\text{Accuracy} = \frac{\#\ \text{correct}}{\#\ \text{correct} + \#\ \text{incorrect}} = (42+36) / 100 = 78\%$$

Elements of the confusion matrix are **not** created equal:

- Suppose "Yes" means an email is spam

Model performance metrics - classification

	Predicted Yes	Predicted No
Observed Yes	42	18
Observed No	4	36

$$\text{Accuracy} = \frac{\#\ \text{correct}}{\#\ \text{correct} + \#\ \text{incorrect}} = (42+36) / 100 = 78\%$$

Elements of the confusion matrix are **not** created equal:

- Suppose "Yes" means an email is spam
- Suppose "Yes" means testing positive for Covid19

Model performance metrics - classification

	Predicted Yes	Predicted No
Observed Yes	42	18
Observed No	4	36

$$\text{Accuracy} = \frac{\#\ \text{correct}}{\#\ \text{correct} + \#\ \text{incorrect}} = (42+36) / 100 = 78\%$$

Elements of the confusion matrix are **not** created equal:

- Suppose "Yes" means an email is spam
- Suppose "Yes" means testing positive for Covid19

Some model performance metrics can get at different components of the confusion matrix

Model performance metrics - classification

	Predicted Yes	Predicted No
Observed Yes	42	18
Observed No	4	36

$$\text{Accuracy} = \frac{\#\ \text{correct}}{\#\ \text{correct} + \#\ \text{incorrect}} = (42+36) / 100 = 78\%$$

Metrics from confusion matrix:
https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/

Metrics for classification:
https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall

Link to ROC and AUC details:
https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc

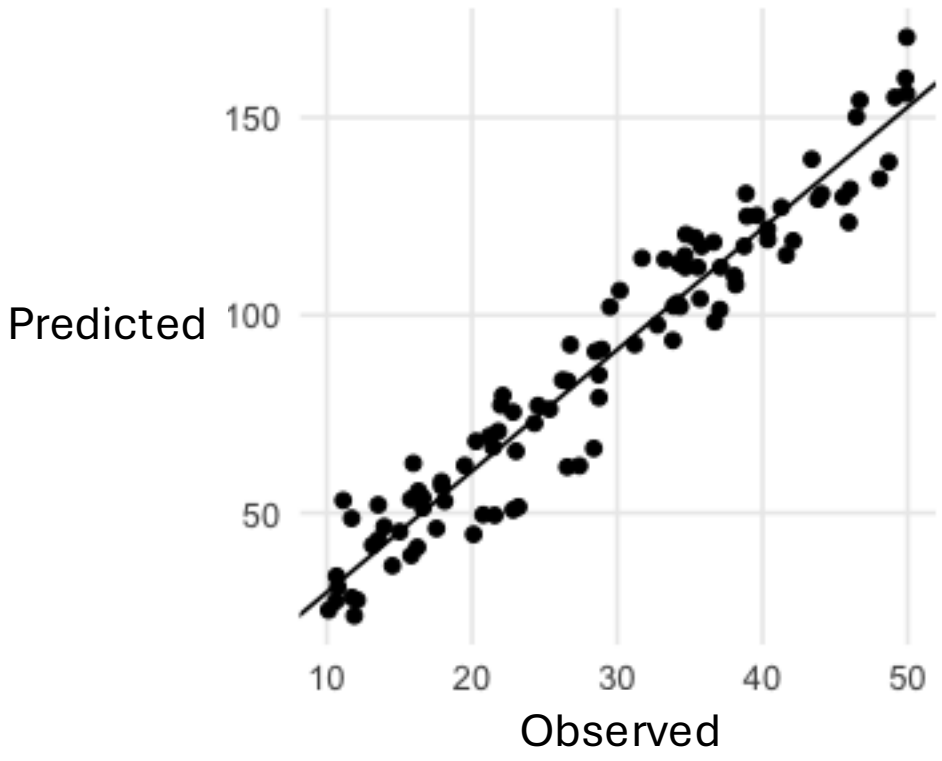
Elements of the confusion matrix are **not** created equal:

- Suppose "Yes" means an email is spam
- Suppose "Yes" means testing positive for Covid19

Some model performance metrics can get at different components of the confusion matrix

Model performance metrics - regression

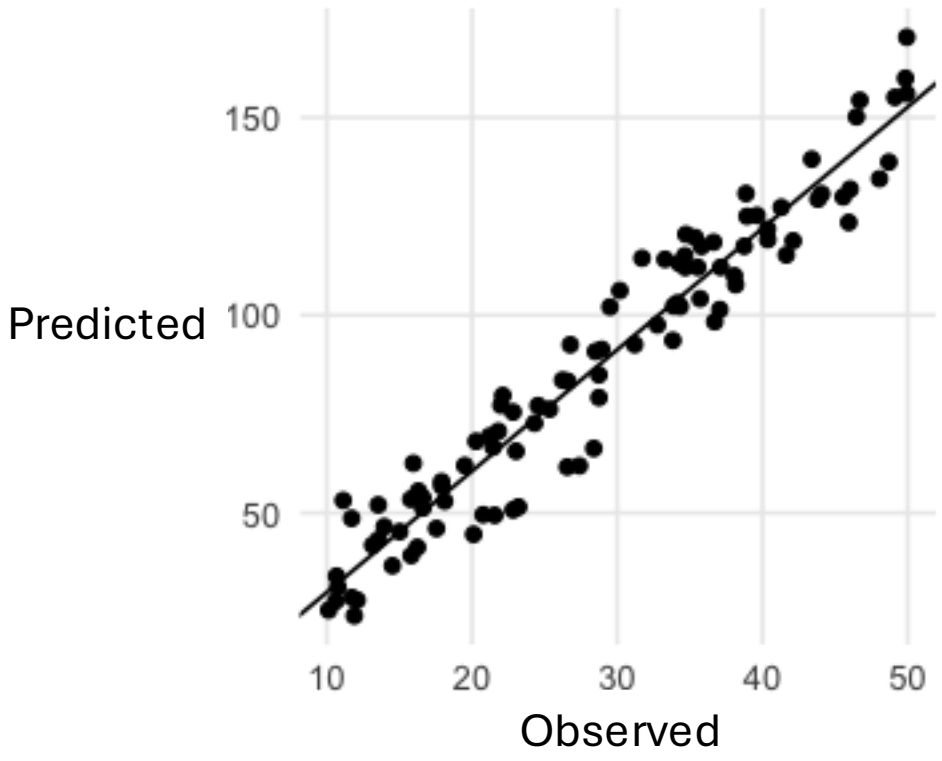
Most ask in some way: **how big are the residuals?**



Model performance metrics - regression

Most ask in some way: **how big are the residuals?**

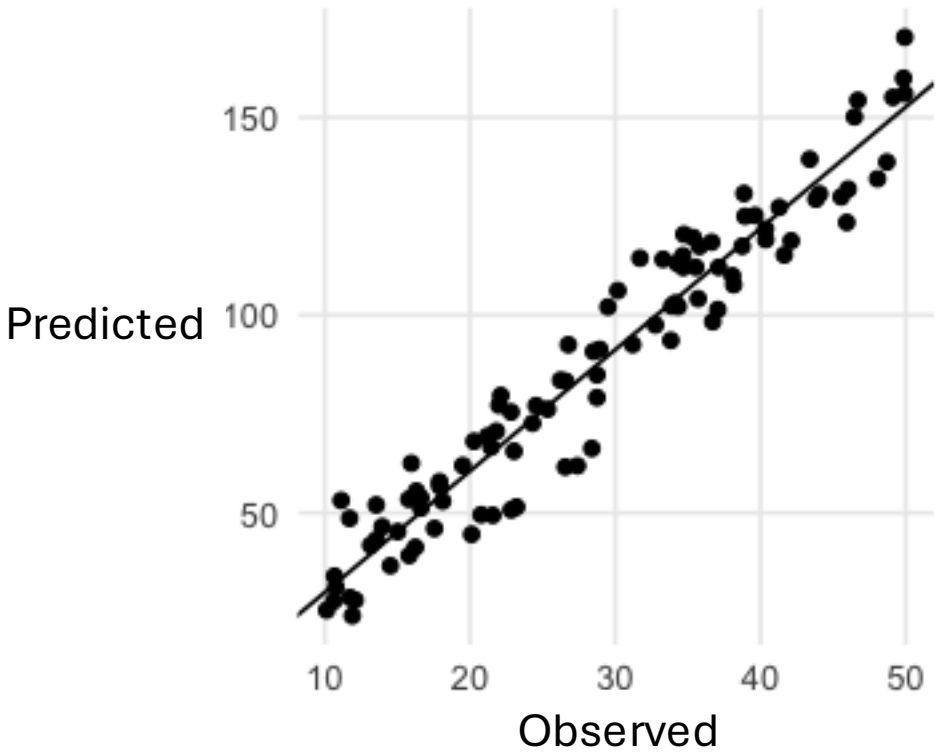
- Root mean squared error (RMSE)
- Mean absolute error
- Many more



Model performance metrics - regression

Most ask in some way: **how big are the residuals?**

- Root mean squared error (RMSE)
- Mean absolute error
- Many more



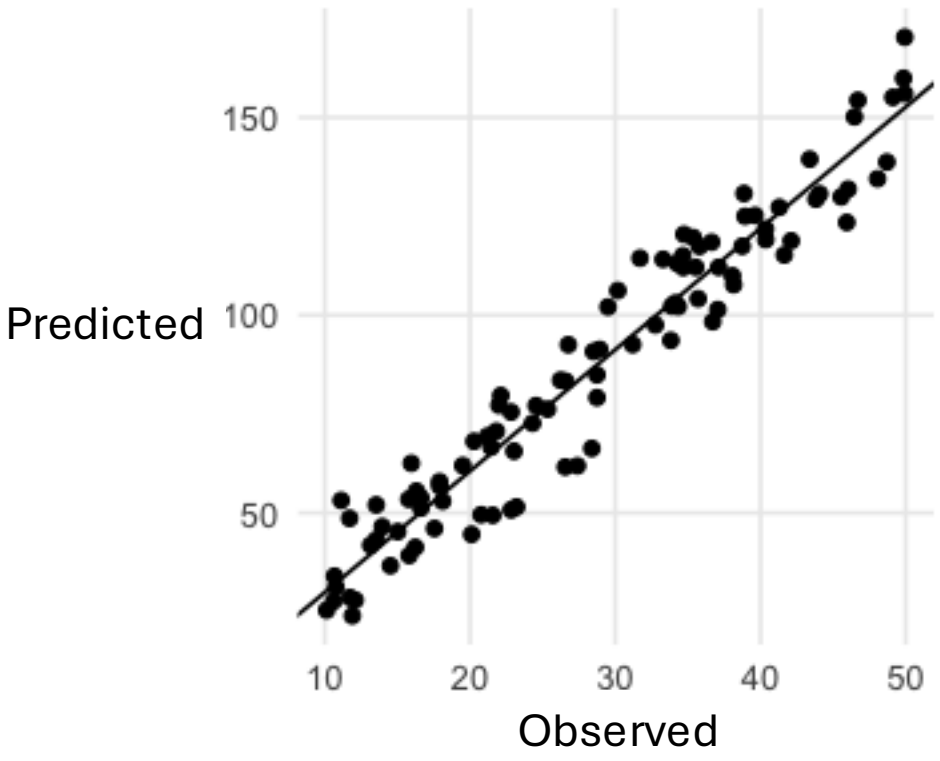
$$RMSE = \frac{\sqrt{\sum(observed - predicted)^2}}{\mathrm{n}}$$

Model performance metrics - regression

Most ask in some way: **how big are the residuals?**

- Root mean squared error (RMSE)
- Mean absolute error
- Many more

Many are related to R^2



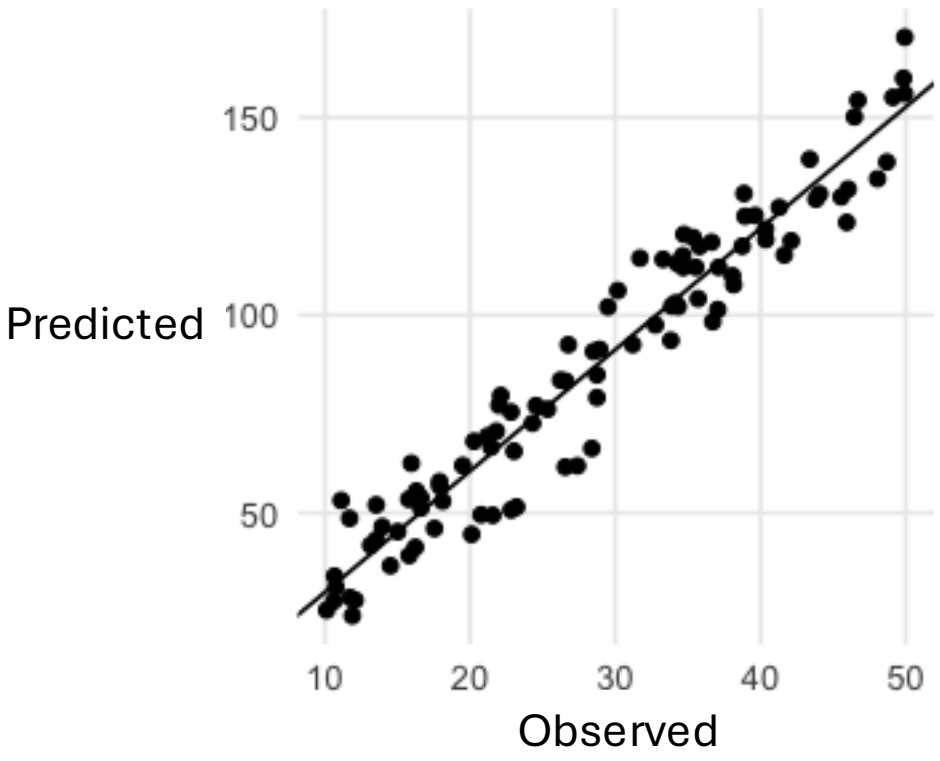
$$RMSE = \frac{\sqrt{\sum(observed - predicted)^2}}{\mathrm{n}}$$

Model performance metrics - regression

Most ask in some way: **how big are the residuals?**

- Root mean squared error (RMSE)
- Mean absolute error
- Many more

Many are related to R^2



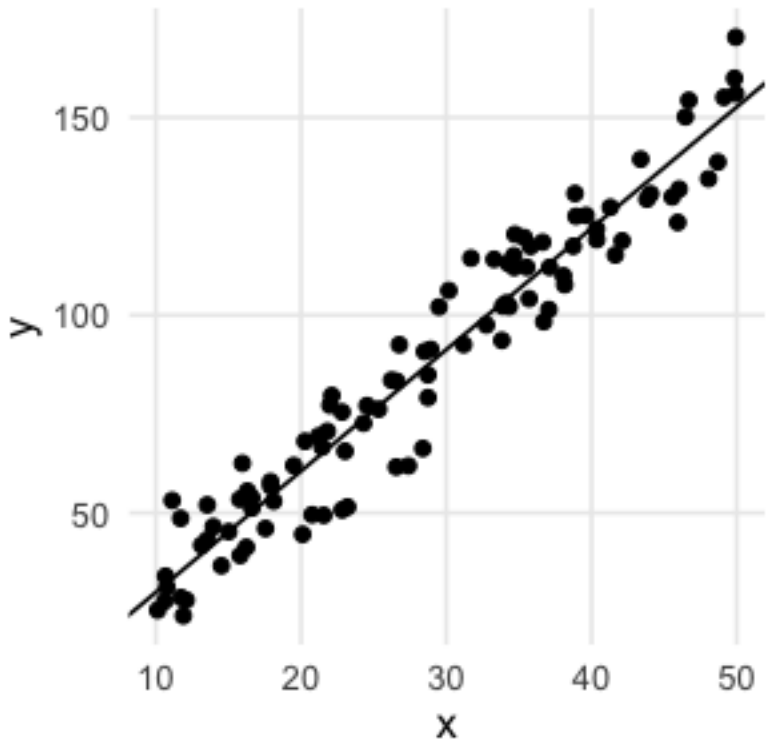
$$RMSE = \frac{\sqrt{\sum(observed - predicted)^2}}{\mathrm{n}}$$

$$\mathrm{R}^2 = 1 - \frac{\mathrm{RMSE}^2}{\mathrm{Var(y)}}$$

RMSE vs R^2
https://datascience.stackexchange.com/questions/100605/rmse-vs-r-squared

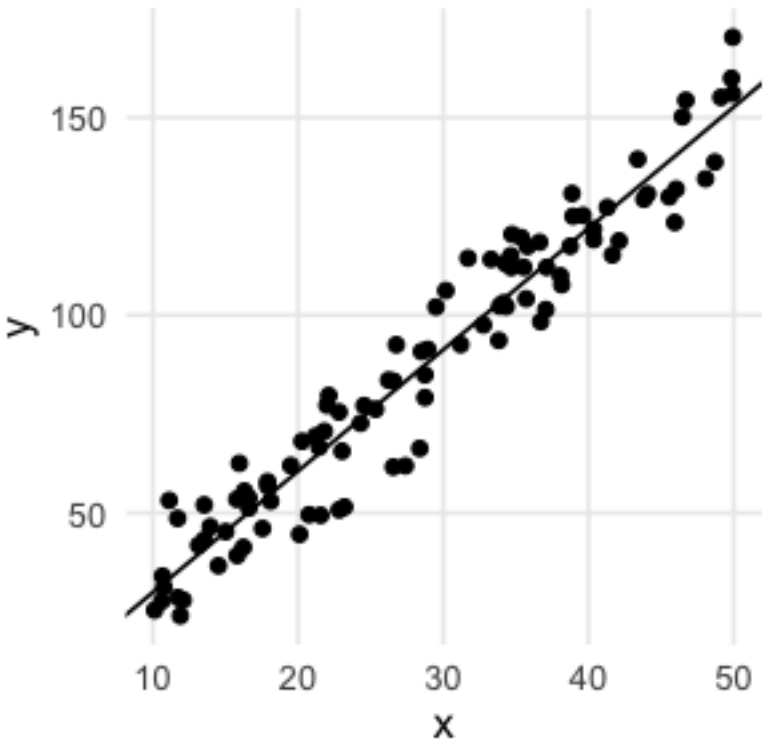
Model performance metrics - regression

- Many metrics compute some sort of “average” to get at overall performance



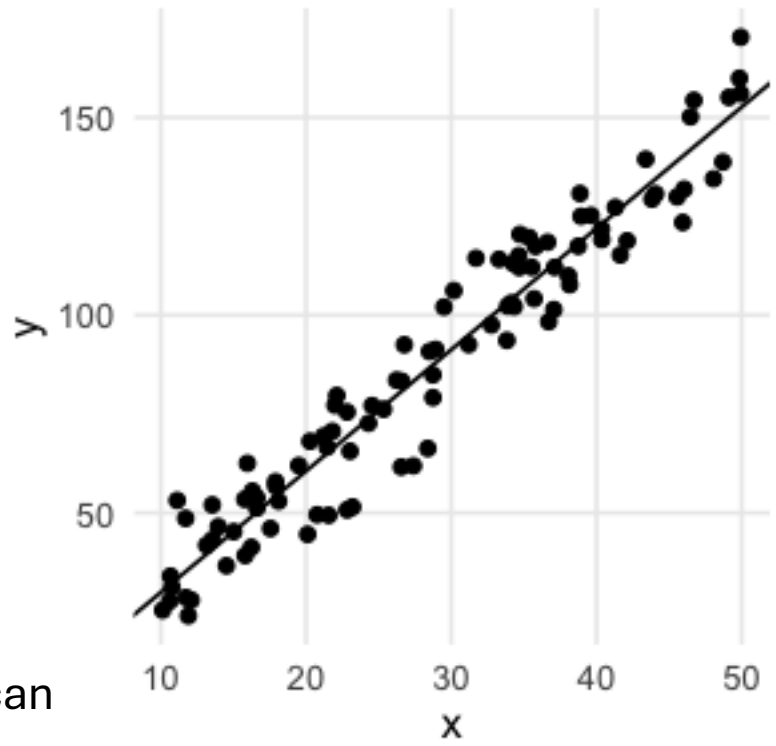
Model performance metrics - regression

- Many metrics compute some sort of “average” to get at overall performance
- Some metrics:
  - Are more sensitive to extremes
  - Consider over-estimation differently than under-estimation
  - Etc.



Model performance metrics - regression

- Many metrics compute some sort of “average” to get at overall performance
- Some metrics:
  - Are more sensitive to extremes
  - Consider over-estimation differently than under-estimation
  - Etc.



Here’s a link with lots of metrics that can be calculated, with some practical examples: https://towardsai.net/p/artificial-intelligence/the-essential-guide-to-ml-evaluation-metrics-for-regression

Model performance metrics – key takeaways

Model performance metrics – key takeaways

Model performance metrics are:

- Diverse and variable
- Able to change your perception of the model
- Deserving of thoughtful consideration – costs and benefits to each decision

Model performance metrics – key takeaways

Model performance metrics are:

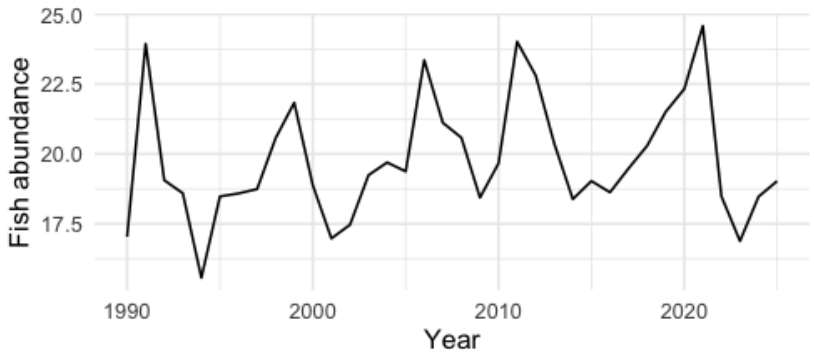
- Diverse and variable
- Able to change your perception of the model
- Deserving of thoughtful consideration – costs and benefits to each decision

Because the model is completely focused on mimicking the outputs, how you measure how well it mimics them is highly important

An example…

We build a model to predict a lake's fish abundance over time based on fishing pressure and weather.

We train the model using data from 1990-2025

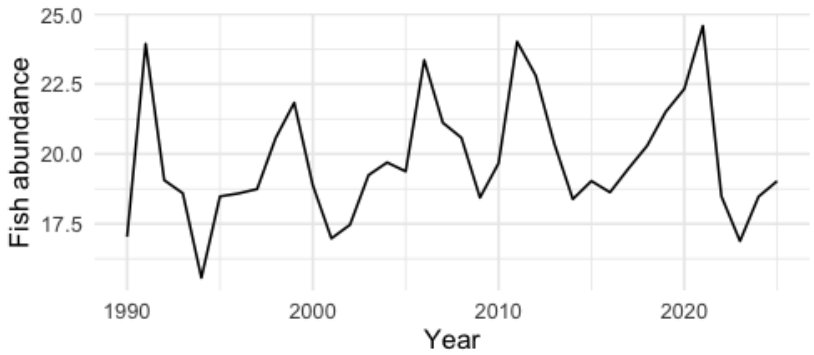


An example…

We build a model to predict a lake's fish abundance over time based on fishing pressure and weather.

We train the model using data from 1990-2025

It predicts those years perfectly.



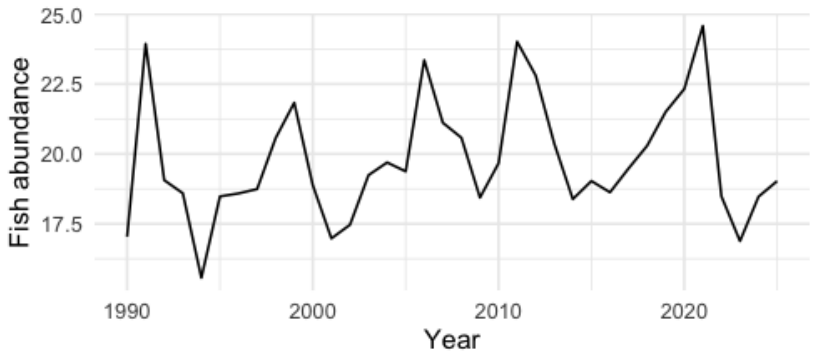
An example…

We build a model to predict a lake's fish abundance over time based on fishing pressure and weather.

We train the model using data from 1990-2025

It predicts those years perfectly.

Is this a good model?



An example…

We build a model to predict a lake's fish abundance over time based on fishing pressure and weather.

We train the model using data from 1990-2025

It predicts those years perfectly.

Is this a good model?

3 sets of data

Training set: used to train the model

Validation set: used by the model/researcher to adjust parameters

Test set: used to evaluate the model performance

3 sets of data

Often handled internally in the code packages that build the models

Training set: used to train the model

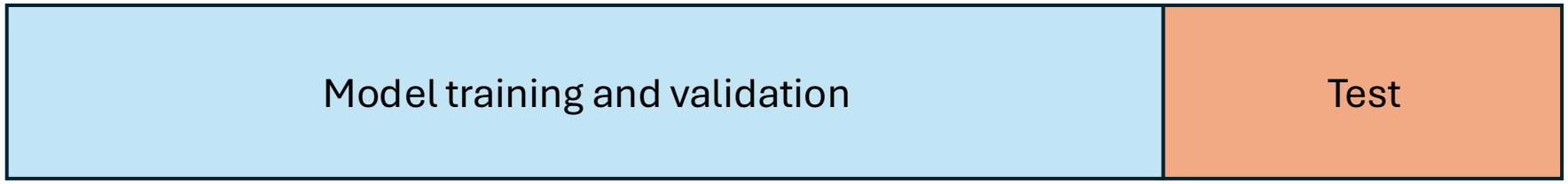
Validation set: used by the model/researcher to adjust parameters

Test set: used to evaluate the model performance

You should *always* evaluate your model with an **independent test set**

The simplest option is to:

- Hold out 20% of your dataset as the **test set**
- Train your model on the 80% **training set**
- Calculate performance metrics on the test set

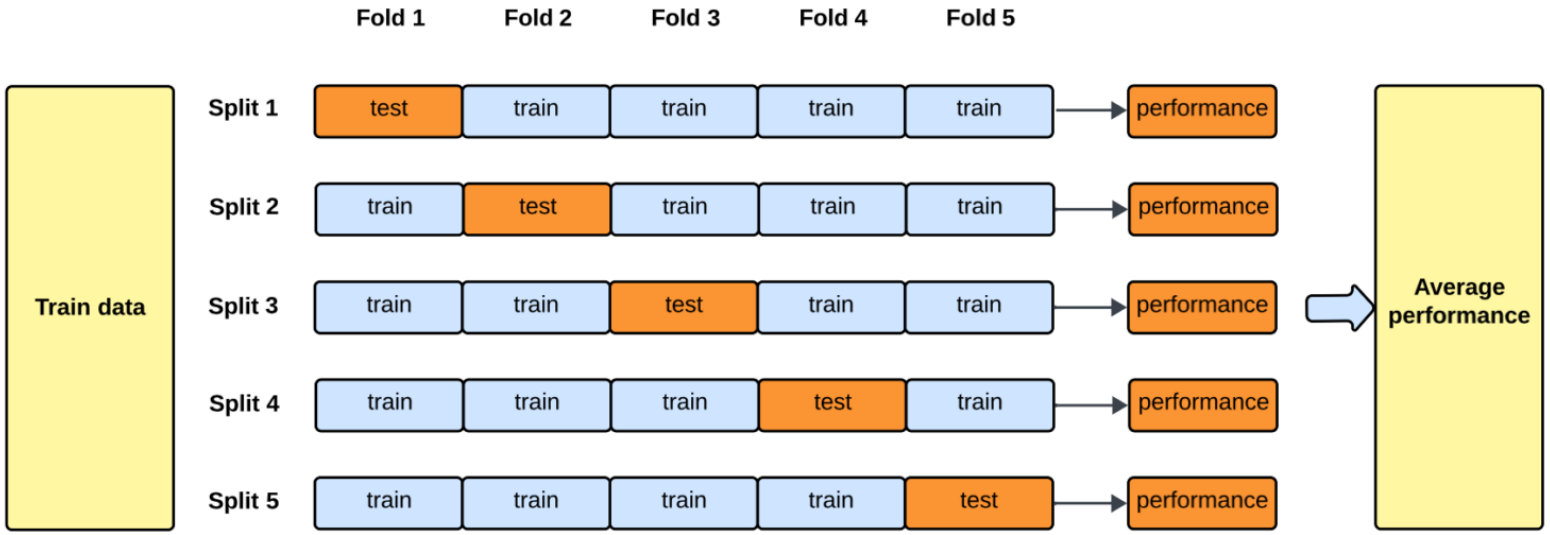


Sometimes, sample size makes this impractical or requires alternatives for a specific application…we'll talk about those later.

K-fold cross-validation

Randomly split your training data into k subsets (“folds”)

For k = 5:



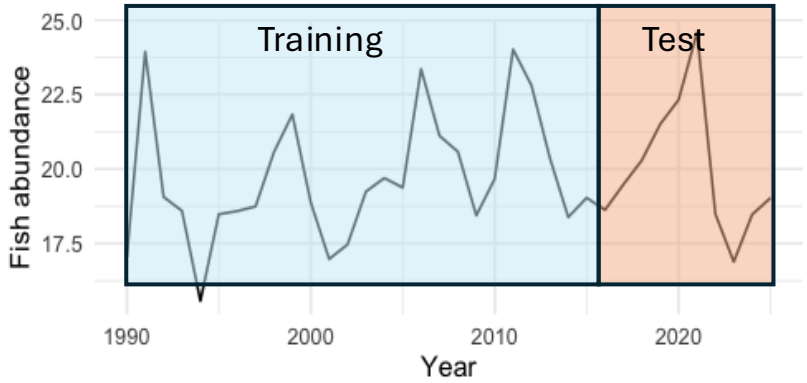
When k = number of rows, it’s a leave-one-out cross validation

https://towardsdatascience.com/how-to-cross-validation-with-time-series-data-9802a06272c6/

Model evaluation should reflect the structure of your prediction problem

Model evaluation should reflect the structure of your prediction problem

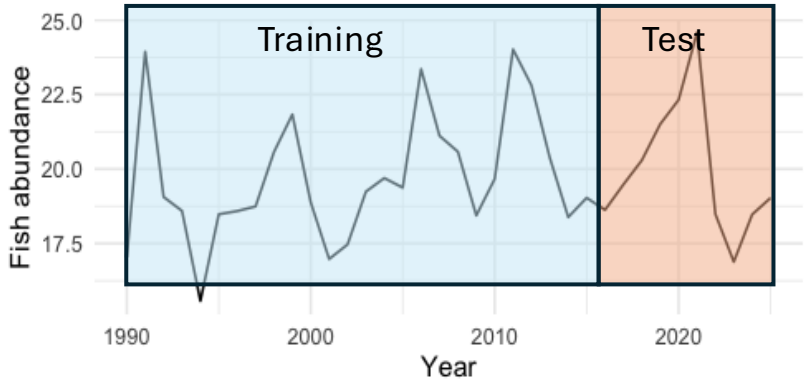
For our fish example, we might be most interested in **predicting the future**



More options for time series:
https://towardsdatascience.com/how-to-cross-validation-with-time-series-data-9802a06272c6/

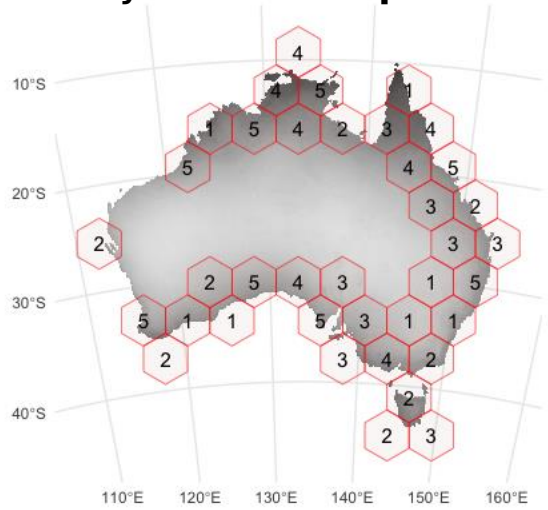
Model evaluation should reflect the structure of your prediction problem

For our fish example, we might be most interested in **predicting the future**



More options for time series: https://towardsdatascience.com/how-to-cross-validation-with-time-series-data-9802a06272c6/

If our prediction problem is spatial, we should probably **consider space** in the model evaluation



blockCV: https://github.com/rvalavi/blockcv

Some thoughts on what constitutes a “final model”

If you’re interested in making predictions outside the available dataset…

Some thoughts on what constitutes a “final model”

If you’re interested in making predictions outside the available dataset…

you probably want to make final predictions using all of the available data…

Some thoughts on what constitutes a “final model”

If you’re interested in making predictions outside the available dataset…

you probably want to make final predictions using all of the available data…

but using all of the data doesn’t leave any available for model evaluation…

Some thoughts on what constitutes a “final model”

If you’re interested in making predictions outside the available dataset…

you probably want to make final predictions using all of the available data…

but using all of the data doesn’t leave any available for model evaluation…

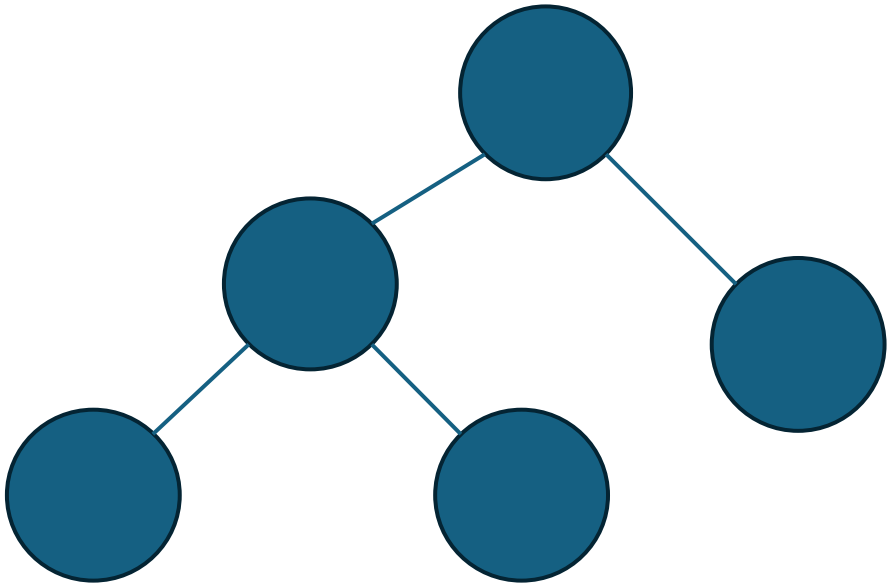
One way that works:

- Build a model using the training set(s)
- Evaluate the model using the test set(s)
- Re-build the model
  - Full dataset
  - Same parameters
  - “Final model”
- Make predictions using the final model

Classification and Regression Trees (CART)

Until a stopping condition is met

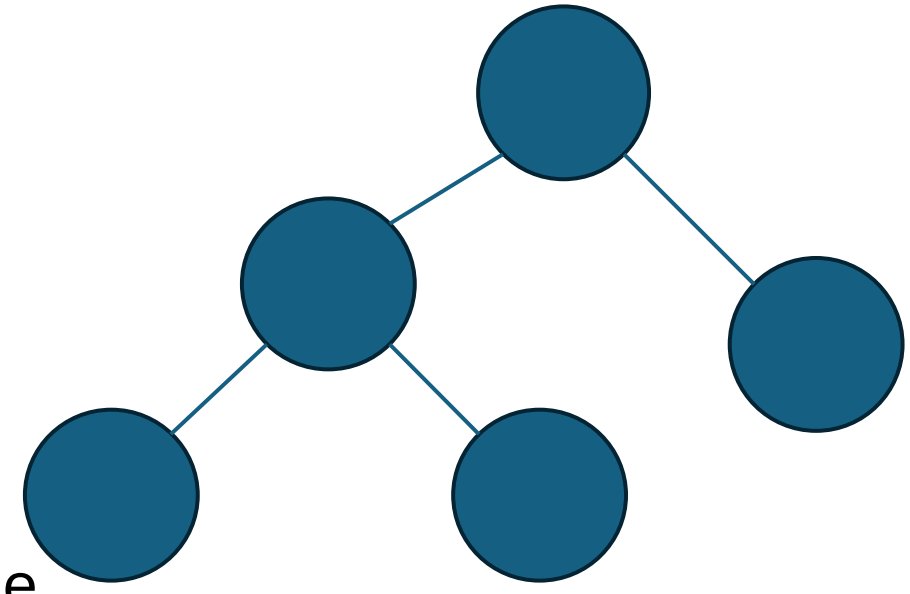
- Evaluate best split for each predictor
- Select the optimal split
- Create new nodes



Classification and Regression Trees (CART)

Until a stopping condition is met

- Evaluate best split for each predictor
- Select the optimal split
- Create new nodes



Stopping conditions:

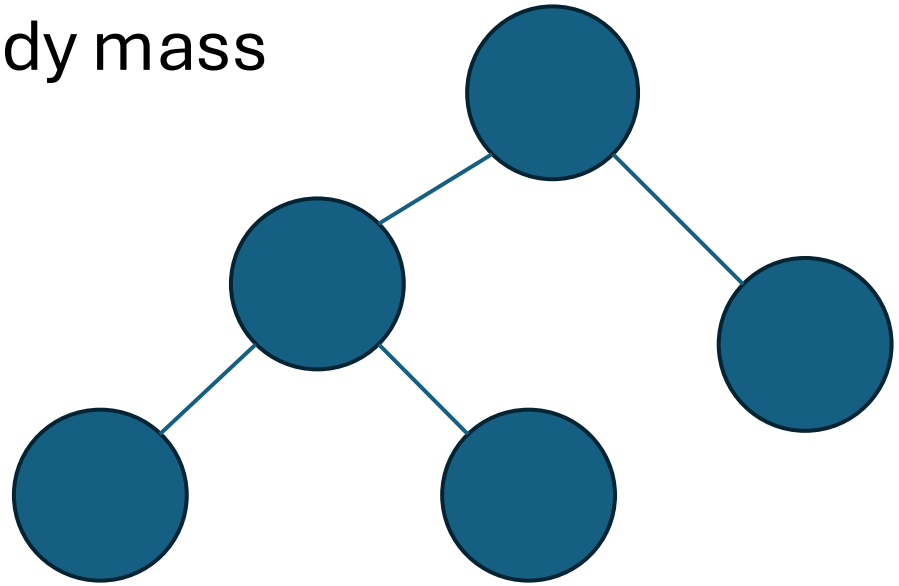
- Maximum tree depth
- Minimum number of samples per node
- No improvement in performance
- Node is all one class / value

Classification and Regression Trees (CART)

Example:

Penguin species ~ flipper length + body mass

Penguin species	Flipper length	Body mass
Adelie	90	80
Gentoo	75	20
Gentoo	77	32
Chinstrap	76	12



Classification and Regression Trees (CART)

Until a stopping condition is met

- Evaluate best split for each predictor
- Select the optimal split
- Create new nodes

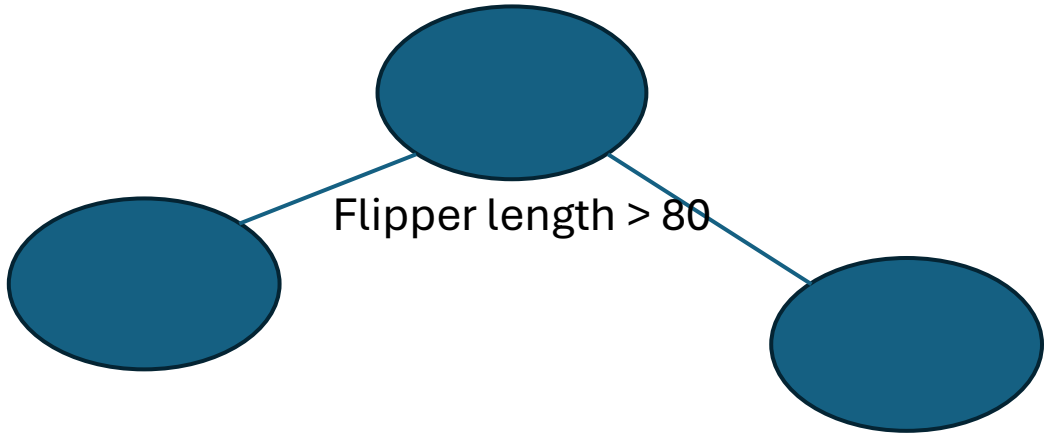
Penguin species	Flipper length	Body mass
Adelie	90	80
Gentoo	75	20
Gentoo	77	32
Chinstrap	76	12

Classification and Regression Trees (CART)

Until a stopping condition is met

- Evaluate best split for each predictor
- Select the optimal split
- Create new nodes

Penguin species	Flipper length	Body mass
Adelie	90	80
Gentoo	75	20
Gentoo	77	32
Chinstrap	76	12

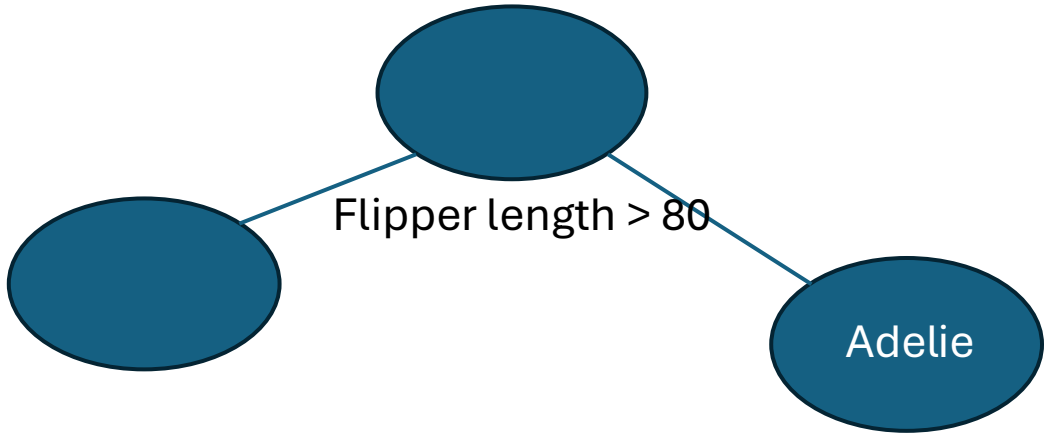


Classification and Regression Trees (CART)

Until a stopping condition is met

- Evaluate best split for each predictor
- Select the optimal split
- Create new nodes

Penguin species	Flipper length	Body mass
Adelie	90	80
Gentoo	75	20
Gentoo	77	32
Chinstrap	76	12

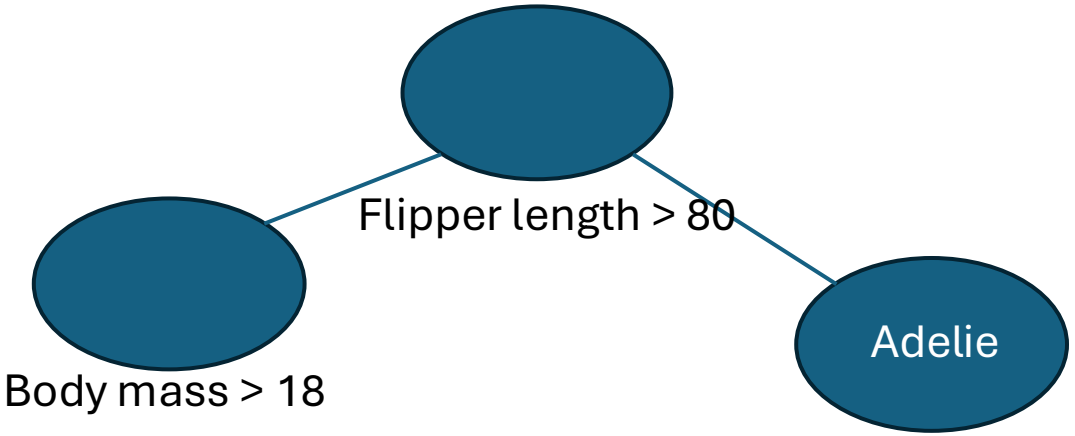


Classification and Regression Trees (CART)

Until a stopping condition is met

- Evaluate best split for each predictor
- Select the optimal split
- Create new nodes

Penguin species	Flipper length	Body mass
Adelie	90	80
Gentoo	75	20
Gentoo	77	32
Chinstrap	76	12

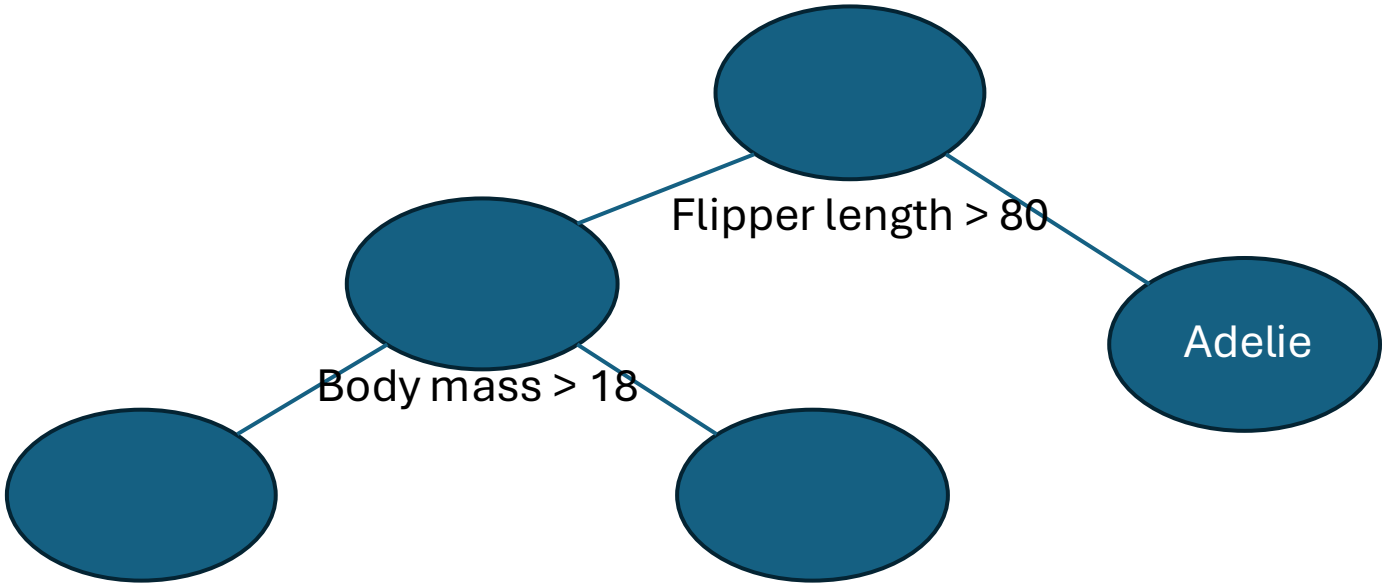


Classification and Regression Trees (CART)

Until a stopping condition is met

- Evaluate best split for each predictor
- Select the optimal split
- Create new nodes

Penguin species	Flipper length	Body mass
Adelie	90	80
Gentoo	75	20
Gentoo	77	32
Chinstrap	76	12

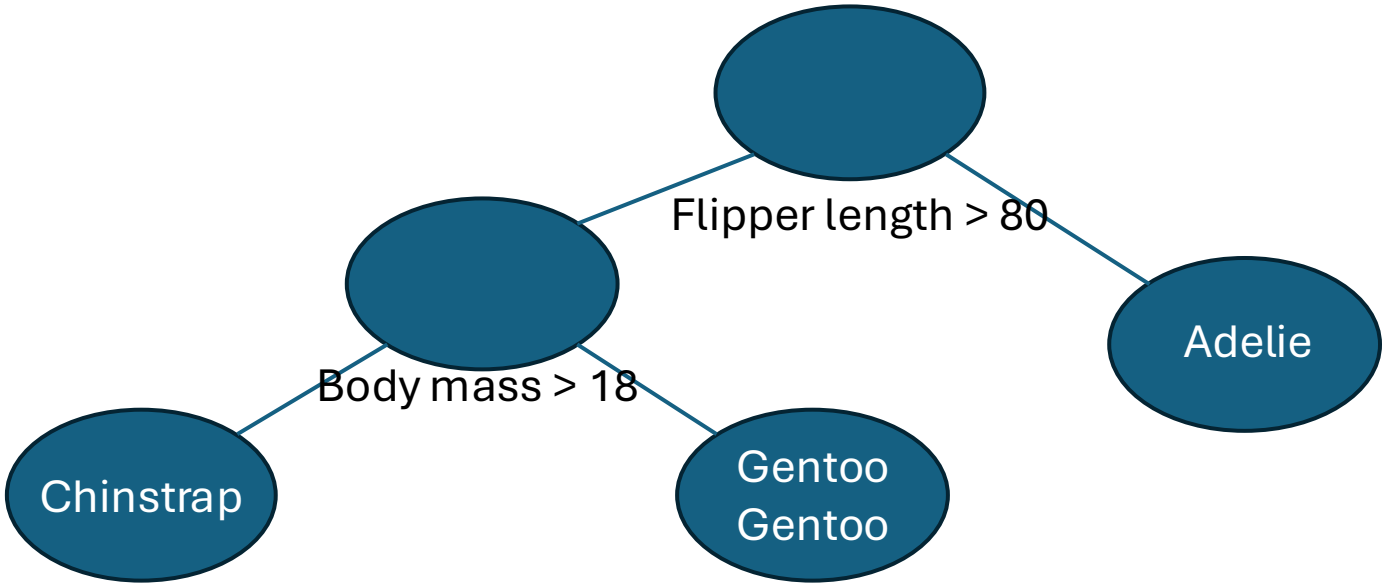


Classification and Regression Trees (CART)

Until a stopping condition is met

- Evaluate best split for each predictor
- Select the optimal split
- Create new nodes

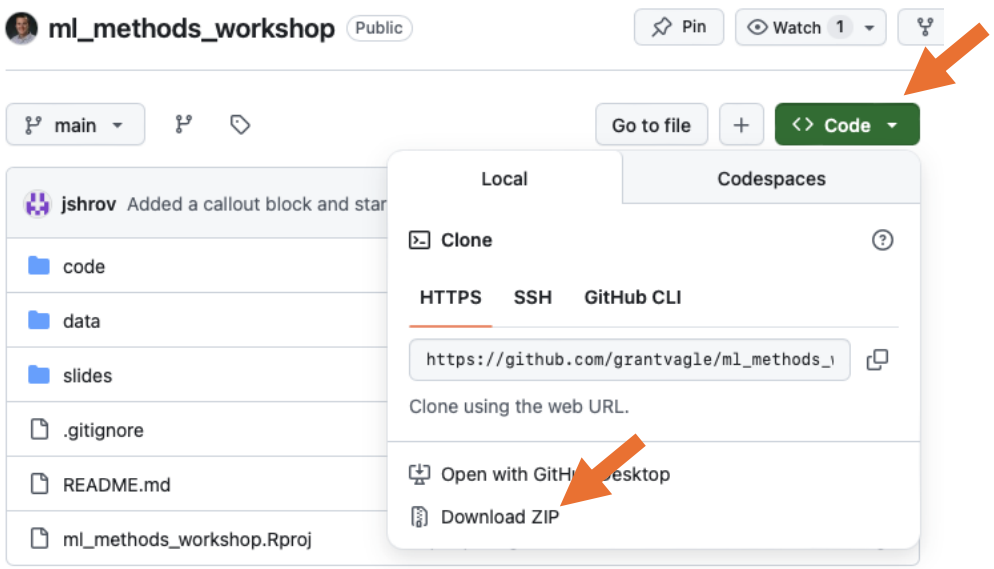
Penguin species	Flipper length	Body mass
Adelie	90	80
Gentoo	75	20
Gentoo	77	32
Chinstrap	76	12

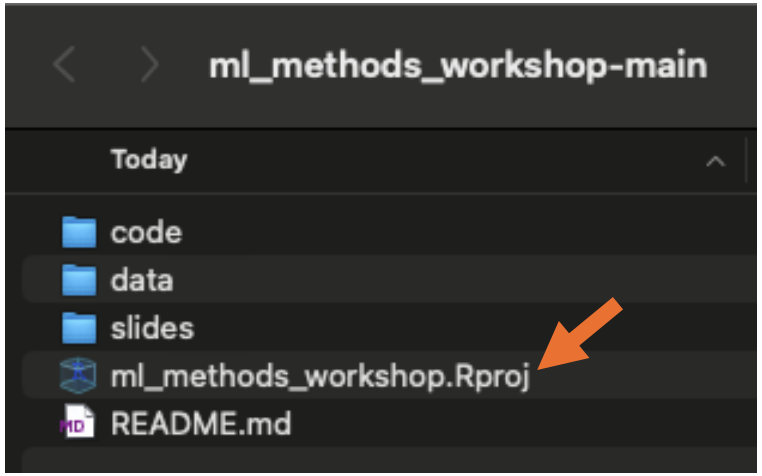


All nodes are “pure” so we stop

Now to a hands-on coding example…

- Go to: https://github.com/grantvagle/ml_methods_workshop
- Click the green “Code” button
- Download the .zip file
- Unzip the folder
- Click to open the .Rproj file





Once you’re set up, work through the **01_cart_examples.qmd** file in the **code** folder